

Python cho Machine Learning: NumPy, Pandas & Visualization

Python & Machine Learning 2026

TA: Lê Trần Tấn Phát

Nền tảng xử lý và trực quan hóa dữ liệu cho Machine Learning

Machine Learning & IoT Lab (ML IoT)

Ho Chi Minh City University of Technology (HCMUT)

1. Vì sao cần scientific Python stack?
2. NumPy: mảng, bộ nhớ, vectorization.
3. Pandas: DataFrame, EDA, giới hạn.
4. Thực hành với Palmer Penguins.
5. Visualization với Matplotlib và Seaborn.
6. Tổng kết.

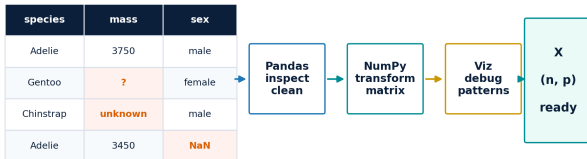
Sau buổi học, sinh viên có thể

- ▶ Giải thích vì sao NumPy và Pandas là nền tảng của ML/DL workflow.
- ▶ Đọc được các khái niệm `shape`, `dtype`, `broadcasting`, `index`, `missing values`.
- ▶ Biết khi nào dùng NumPy, khi nào dùng Pandas, và khi nào cần công cụ khác.
- ▶ Dùng `visualization` để kiểm tra dữ liệu trước khi modeling.

Trước khi train model, ta có gì?

Đi từ bảng dữ liệu lộn xộn đến ma trận số sẵn sàng cho model

RAW TABLE



mixed types • missing values • labels

from messy context to model-ready numeric data

Dữ liệu thực tế

- ▶ Nhiều dòng, nhiều cột.
- ▶ Có missing values.
- ▶ Có số, chữ, ngày tháng, category.
- ▶ Có outlier và lỗi nhập liệu.

Model cần gì?

- ▶ Ma trận số rõ shape.
- ▶ Feature được chuẩn hóa.
- ▶ Label và input đúng kiểu.
- ▶ Pipeline có thể lặp lại.

Thư viện / công cụ	Vai trò
NumPy	Tính toán số trên mảng nhiều chiều.
Pandas	Đọc, làm sạch, phân tích dữ liệu dạng bảng.
Matplotlib	Nền tảng vẽ biểu đồ và tùy biến chi tiết.
Seaborn	Visualization thống kê nhanh trên DataFrame.
Scikit-learn	Machine learning truyền thống.
PyTorch/TensorFlow	Deep learning, tensor, GPU.

Raw data → Load → Clean → Transform → Visualize → Model

Pandas

Dữ liệu còn nhiều ngữ cảnh: tên cột, category, missing values, bảng.

NumPy

Dữ liệu đã gán với dạng số: vector, matrix, tensor.

Visualization

Kiểm tra phân phối, outlier, quan hệ, và lỗi dữ liệu.

Python list có đủ cho dữ liệu lớn?

Cùng một phép toán, hai cách mô tả

```
values = [1, 2, 3, 4]
[x * 2 for x in values]      # Python loop
arr = np.array(values)
arr * 2                      # array operation
```

Python list

- ▶ Linh hoạt, chứa nhiều kiểu dữ liệu.
- ▶ Mỗi phần tử là object riêng.
- ▶ Không tối ưu cho tính toán số hàng loạt.

```
values = [1, "hello", 3.14]  
type(values)    # list
```

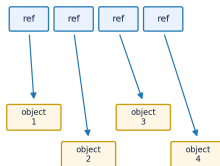
NumPy array

- ▶ Đồng nhất kiểu dữ liệu.
- ▶ Có shape và dtype rõ ràng.
- ▶ Tối ưu cho phép toán vector/matrix.

```
arr = np.array([1, 2, 3, 4])  
arr.shape, arr.dtype    # (4,), int64
```

- ▶ Python list lưu tham chiếu tới nhiều object.
- ▶ NumPy array lưu dữ liệu trong typed buffer liên tục.
- ▶ Dữ liệu liền nhau giúp CPU cache hoạt động hiệu quả hơn.

Python list



Flexible, but each number is a Python object.
More pointer chasing, more overhead.

NumPy array

`dtype=float32 | shape=(8,)`



Contiguous typed buffer

Same dtype, compact memory layout.
Vectorized operations can run in optimized native code.

1. Contiguous memory

Dữ liệu số được lưu gần nhau, giảm overhead truy cập object.

2. Vectorized operations

Một lệnh mô tả phép toán trên cả mảng, giảm vòng lặp Python.

3. Native code + BLAS backend

Phép toán nặng chạy qua C/Fortran và BLAS/LAPACK; backend thường gặp là OpenBLAS hoặc Intel MKL.

```
C = A @ B      # matrix multiplication delegates to optimized native code
```

Dữ liệu	Shape
Vector feature	(F,)
Bảng dữ liệu	(N, F)
Ảnh màu	(H, W, C)
Batch ảnh	(B, H, W, C)

```
X = np.zeros((100, 5))
img = np.zeros((224, 224, 3))
batch = np.zeros((32, 224, 224, 3))
X.shape      # (100, 5)
batch.shape  # (32, 224, 224, 3)
```

Trước phép toán quan trọng, hãy đọc shape: (100, 5) @ (5,) -> (100,).

- ▶ float64: chính xác hơn, tốn bộ nhớ hơn.
- ▶ float32: phổ biến trong deep learning.
- ▶ uint8: pixel ảnh trong khoảng 0–255.
- ▶ int32/int64: index, count, encoded category.

```
a = np.ones((1000, 1000), dtype=np.float32)
a.nbytes / 1e6      # 4.0 MB
```

Vì sao cần quan tâm?

- ▶ Ảnh hưởng memory footprint.
- ▶ Ảnh hưởng tốc độ tính toán.
- ▶ Ảnh hưởng tương thích với model.
- ▶ Dễ tạo bug ngầm nếu dtype sai.

```
img = np.array([0, 128, 255], dtype=np.uint8)
img = img.astype(np.float32) / 255
```

Tư duy vòng lặp

Điều khiển từng phần tử trong Python.

```
normalized = []  
for x in scores:  
    normalized.append((x - mean) / std)
```

Tư duy array

Mô tả trực tiếp phép toán trên cả mảng.

```
normalized = (scores - scores.mean())  
            / scores.std()
```

- ▶ Gắn công thức toán hơn.
- ▶ Ít code và thường nhanh hơn.

Với ma trận X, chuẩn hóa từng feature cần `mean(axis=0)` và `std(axis=0)`.

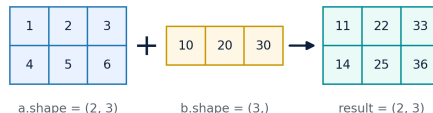
Quy tắc

NumPy so sánh shape từ phải sang trái. Ở mỗi chiều, hai kích thước phải bằng nhau hoặc một bên bằng 1. Chiều bị thiếu ở bên trái được xem như 1.

- ▶ $(2, 3) + (3,) \rightarrow (2, 3)$: số 3 khớp; vector được áp dụng cho từng hàng.
- ▶ $(2, 3) + (2,)$ gây lỗi: chiều cuối 3 và 2 không tương thích.

```
X = np.random.randn(100, 5)
mu = X.mean(axis=0)    # (5,)
X_centered = X - mu
```

Broadcasting aligns trailing dimensions



The vector is reused across rows; no manual tiling is needed.

View

Nhìn vào cùng vùng bộ nhớ với array gốc. Sửa view có thể sửa array gốc.

Copy

Tạo vùng bộ nhớ mới. Sửa copy không ảnh hưởng array gốc.

```
a = np.array([1, 2, 3, 4])    b = a[1:3]
b[0] = 99                    # a becomes [1, 99, 3, 4]
c = a[1:3].copy()            # independent memory
```

Khi cần dữ liệu độc lập để mutate, hãy `copy()` tường minh.

NumPy mạnh khi

- ▶ Dữ liệu đã là numeric.
- ▶ Cần tính vector/matrix.
- ▶ Cần tốc độ và memory rõ ràng.

Cần cẩn thận với

- ▶ Shape: kiểm tra `arr.shape`.
- ▶ Dtype: kiểm tra `arr.dtype`.
- ▶ Broadcasting: viết rõ `axis/reshape`.
- ▶ Mutate độc lập: dùng `copy()`.

Trong notebook, sinh viên sẽ

- ▶ Tạo array và đọc `shape`, `dtype`, `ndim`.
- ▶ Thực hành indexing, slicing, boolean mask.
- ▶ So sánh loop thinking và vectorized thinking.
- ▶ Thực hành broadcasting trong feature scaling.
- ▶ Nhận diện view/copy qua vài tình huống nhỏ.

Code được chạy trong notebook đi kèm.

- ▶ Khi dữ liệu còn là bảng nhiều kiểu: text, category, datetime, missing values.
- ▶ Khi cần join, groupby, pivot, xử lý CSV/Excel/SQL.
- ▶ Khi cần lưu ngữ cảnh cột và dòng rõ ràng.
- ▶ Khi dữ liệu quá lớn cho RAM.
- ▶ Khi cần tensor GPU và autograd cho deep learning.

Câu hỏi tiếp theo

Nếu dữ liệu nằm trong CSV với tên cột, category, ngày tháng và missing values, ta nên làm gì trước?

Dữ liệu không phải lúc nào cũng sạch

Một bảng “đọc được” chưa chắc đã sẵn sàng để phân tích

A table can look readable and still need inspection

Palmer Penguins: real columns, mixed dtypes, missing values

	species	island	bill_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	3750.0	male
1	Adelie	Torgersen	39.5	3800.0	female
2	Adelie	Torgersen	NaN	3250.0	female
3	Adelie	Torgersen	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	3450.0	NaN
5	Adelie	Torgersen	39.3	3650.0	male

(344, 8)

shape

numeric + object

dtypes

19 cells

missing

Dữ liệu thực tế

- ▶ CSV, Excel, SQL table.
- ▶ Có tên cột và nhiều kiểu dữ liệu.
- ▶ Có missing values; cần group, merge, reshape.

Pandas cung cấp

- ▶ Series: một cột có index.
- ▶ DataFrame: bảng có index và columns.
- ▶ API cho EDA và feature engineering.

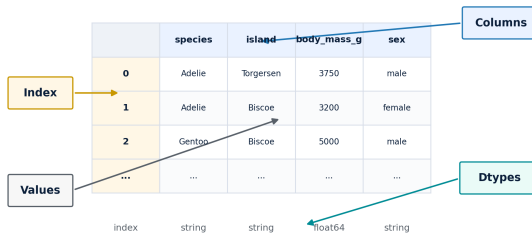
```
df = pd.DataFrame({"species": ["Adelie", "Gentoo"], "body_mass_g": [3750, 5000]})  
df["species"]      # one Series      df.shape      # (2, 2)
```

- ▶ **Index:** nhãn của dòng.
- ▶ **Columns:** tên biến/feature.
- ▶ **Values:** dữ liệu thực tế.
- ▶ **Dtypes:** schema của từng cột.

Không chỉ là matrix

DataFrame giữ cả dữ liệu lẫn ngữ cảnh của bảng.

DataFrame anatomy



The diagram shows a DataFrame table with the following structure:

	species	island	body_mass_g	sex
0	Adelie	Torgersen	3750	male
1	Adelie	Biscoe	3200	female
2	Gentoo	Biscoe	5000	male
...	...	...	...	...

Labels and arrows in the diagram:

- Index:** Points to the first column (index).
- Columns:** Points to the header row (species, island, body_mass_g, sex).
- Values:** Points to the data rows (0, 1, 2, ...).
- Dtypes:** Points to the data types (string, string, float64, string) listed below the table.

- ▶ Mỗi cột DataFrame thường là một array hoặc extension array.
- ▶ Pandas thêm label, index alignment, dtype management và API bảng.
- ▶ DataFrame không phải một ma trận đồng nhất như NumPy array.
- ▶ Điều này giúp Pandas linh hoạt hơn nhưng cũng có overhead.

Cách đọc đúng

NumPy là vùng tính toán số. Pandas là bảng dữ liệu có ngữ cảnh.

Pandas hiện đại cũng hỗ trợ extension arrays và PyArrow backend; không phải mọi cột đều bắt buộc lưu bằng NumPy.

Sức mạnh

- ▶ Tính toán theo label.
- ▶ An toàn hơn khi dữ liệu khác thứ tự.
- ▶ Hữu ích khi join và align.

Nhầm lẫn thường gặp

loc: theo label.
iloc: theo vị trí.

```
df = pd.DataFrame({"score": [85, 92, 78]},  
                  index=[10, 20, 30])  
  
df.iloc[0]      # first row; label 10  
df.loc[10]      # row whose label is 10  
df.loc[0]       # KeyError
```

Missing không chỉ là ô trống

Missing value có thể là lỗi thu thập dữ liệu, hoặc là tín hiệu có ý nghĩa.

- ▶ Detect: `isna`, `notna`.
- ▶ Quantify: số lượng và tỷ lệ missing.
- ▶ Decide: drop, fill, hoặc tạo indicator.

```
df.isna().sum()
clean = df.dropna(subset=["body_mass_g"])
median = df["body_mass_g"].median()
filled = df["body_mass_g"].fillna(median)
```

Câu hỏi cần trả lời

Missing do đâu? Có bao nhiêu? Bỏ dòng có làm lệch dữ liệu không?

Drop/fill là quyết định theo ngữ cảnh, không phải công thức mặc định.

- ▶ Dataset lớn có thể tốn RAM nhanh.
- ▶ Lặp từng dòng thường chậm.
- ▶ Nhiều thao tác chủ yếu chạy một luồng.
- ▶ Copy/view dễ gây nhầm lẫn.

Quy tắc thực dụng

Rất tốt cho EDA và prototype khi dữ liệu vừa trong RAM.

object có thể che lấp

Một chuỗi lạ có thể khiến cả cột số bị đọc thành text.

```
age = pd.Series(["20", "unknown", "22"])
age.dtype      # object
age = pd.to_numeric(age, errors="coerce")
age.isna().sum() # 1
```

Vấn đề

Trong một số tình huống filter rồi sửa, Pandas có thể khiến người học khó đoán đâu là view và đâu là copy.

Copy-on-Write

Pandas 2.x hỗ trợ Copy-on-Write, hướng tới mô hình an toàn hơn: dữ liệu chỉ được copy khi cần ghi.

Best practice: nếu muốn tạo DataFrame độc lập để sửa, copy tường minh.

- ▶ PyArrow giúp lưu dữ liệu columnar hiệu quả hơn.
- ▶ String và nullable dtype tốt hơn so với object truyền thống.
- ▶ Parquet là format hiệu quả hơn CSV cho dữ liệu lớn và typed data.
- ▶ Đây là hướng phát triển quan trọng của Pandas hiện đại.

```
df = pd.read_csv("penguins.csv", dtype_backend="pyarrow")
```

Tình huống	Công cụ cân nhắc
Query dữ liệu trong database	SQL
DataFrame nhanh hơn	Polars
Dữ liệu lớn hơn RAM	Dask
Big data phân tán	Spark
Tensor/GPU/deep learning	PyTorch/TensorFlow

Cùng phân tích Palmer Penguins

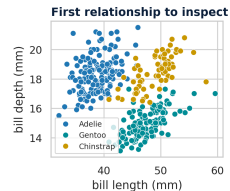
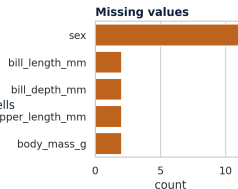
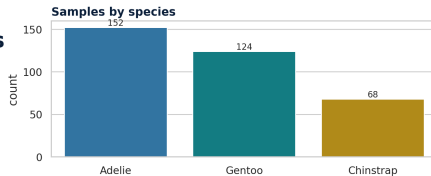
Một dataset nhỏ để luyện EDA bằng Pandas

- ▶ 344 mẫu chim cánh cụt.
- ▶ 3 loài: Adelie, Chinstrap, Gentoo.
- ▶ 3 đảo: Biscoe, Dream, Torgersen.
- ▶ Có numeric, categorical và missing values.

Palmer Penguins

A compact EDA dataset

344	Rows
8	Columns
3	Species
19	Missing cells



Bốn kiểm tra đầu tiên

- ▶ Kích thước bảng.
- ▶ Vài dòng đầu.
- ▶ Dtype từng cột.
- ▶ Missing theo cột.

```
df = pd.read_csv("data/penguins.csv")
df.shape
df.head()
df.dtypes
df.isna().sum()
```

Câu hỏi kiểm tra

Cột nào numeric/categorical? Missing nằm ở đâu?

Một workflow phổ biến

1. Chọn cột cần dùng.
2. Lọc hàng theo điều kiện.
3. Sắp xếp để inspect.

Kết quả vẫn là DataFrame và có thể nối tiếp bằng method chaining.

```
cols = ["species", "body_mass_g"]  
gentoo = df.loc[df["species"] == "Gentoo", cols]  
gentoo = gentoo.sort_values(  
    "body_mass_g", ascending=False  
)
```

Cleaning

- ▶ Kiểm tra missing values.
- ▶ Tạo bản clean cho phân tích cần đủ dữ liệu.
- ▶ Tránh chained assignment.

Feature mới

- ▶ Body mass theo kg.
- ▶ Bill length/depth ratio.
- ▶ Flipper-to-mass ratio.

GroupBy trả lời câu hỏi theo nhóm

► Loài nào có body mass trung bình lớn nhất?

► Flipper length khác nhau theo loài thế nào?

```
df.groupby("species")["body_mass_g"].mean()
```

► Body mass khác nhau theo species và sex ra sao?

► Số mẫu trên từng đảo có cân bằng không?

GroupBy summary: species × sex

One aggregation can answer several comparison questions.

species	sex	n	mean mass (g)	mean flipper (mm)
Adelie	female	73	3369	187.8
Adelie	male	73	4043	192.4
Chinstrap	female	34	3527	191.7
Chinstrap	male	34	3939	199.9
Gentoo	female	58	4680	212.7
Gentoo	male	61	5485	221.5

Pivot

Tạo bảng đếm species theo island để nhìn cấu trúc dataset.

Merge

Nối metadata nhỏ về species vào bảng chính.

Export

Lưu bản clean để dùng cho phần visualization.

Trong notebook, sinh viên sẽ

- ▶ Load data/penguins.csv.
- ▶ Inspect schema và missing values.
- ▶ Clean dữ liệu và tạo feature mới.
- ▶ GroupBy theo species, sex, island.
- ▶ Merge metadata species.
- ▶ Export dữ liệu clean cho visualization.

Code được chạy trong notebook đi kèm.

Hay gặp

- ▶ Chained assignment.
- ▶ Quên reset index sau filter.
- ▶ Dùng `apply(axis=1)` quá sớm.
- ▶ Không kiểm tra dtype sau khi load.

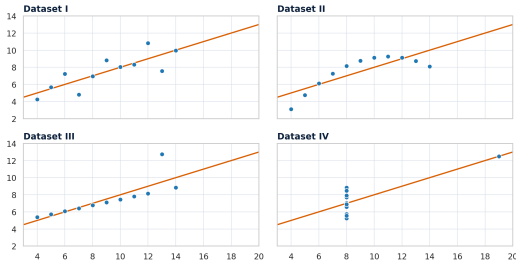
Thói quen tốt

- ▶ Inspect trước khi xử lý.
- ▶ Copy tường minh khi cần mutate.
- ▶ Ưu tiên vectorized operations.
- ▶ Check shape sau merge.

Bạn đã thật sự nhìn dữ liệu chưa?

Bốn dataset có thống kê gần như giống nhau, nhưng cấu trúc hoàn toàn khác

Anscombe's Quartet



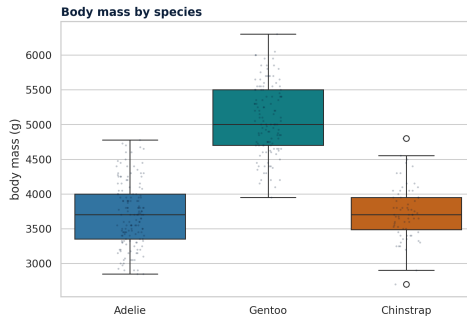
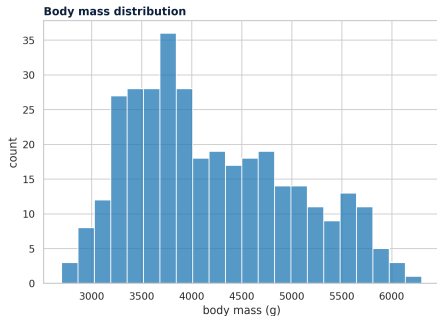
Same mean, variance, correlation and regression line — very different data.

- ▶ Summary statistics không cho thấy đầy đủ hình dạng dữ liệu.
- ▶ Biểu đồ giúp phát hiện outlier, skew, clusters, imbalance.
- ▶ Visualization là công cụ debug dữ liệu trước modeling.
- ▶ Một biểu đồ tốt phải trả lời một câu hỏi cụ thể.

Anscombe's Quartet

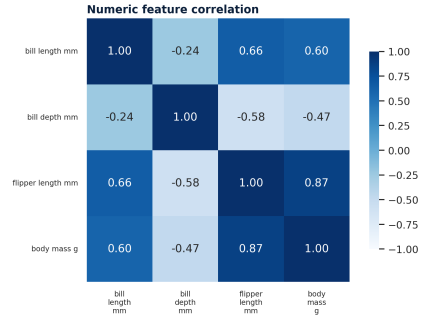
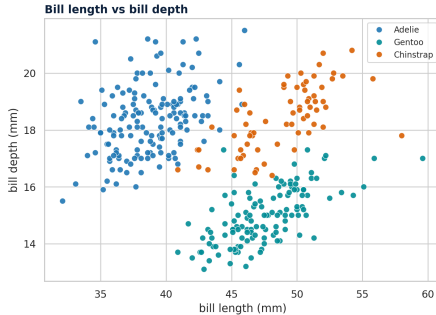
Cùng mean, variance, correlation và đường hồi quy gần như giống nhau; biểu đồ cho thấy bốn cấu trúc khác biệt.

Nếu chưa nhìn dữ liệu, đừng vội train model.



Câu hỏi

Body mass phân phối ra sao, và khác nhau thế nào giữa các species?



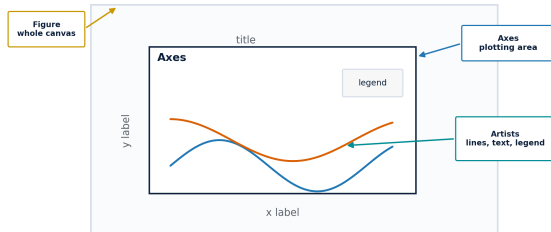
Câu hỏi

Các feature numeric có tách nhóm hoặc tương quan rõ ràng không?

Dùng khi cần

- ▶ Nền tảng rendering plot trong Python.
- ▶ Control chi tiết figure, axes, label, annotation.
- ▶ Phù hợp khi cần tùy biến sâu.

```
fig, ax = plt.subplots()
ax.plot(x, y, label="signal")
ax.set_title("My chart")
ax.set_xlabel("x")
ax.legend()
```

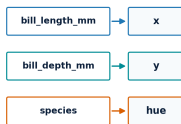


Dùng khi cần

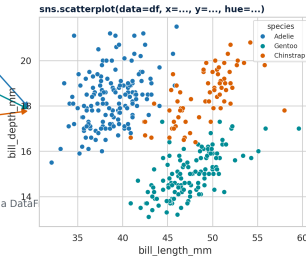
- ▶ Trực quan hóa thống kê trên DataFrame.
- ▶ API gọn cho EDA.
- ▶ Dễ map biến vào x, y, hue, row, col.

```
sns.scatterplot(
    data=df,
    x="bill_length_mm",
    y="bill_depth_mm",
    hue="species"
)
```

DataFrame columns



Seaborn is useful when data already lives in a DataFrame



Câu hỏi	Chart phù hợp
Một biến numeric phân phối ra sao?	Histogram, KDE
Có outlier theo nhóm không?	Boxplot, violin plot
Hai biến numeric liên quan thế nào?	Scatter plot
Target/category có mất cân bằng không?	Count plot
Các feature numeric tương quan ra sao?	Heatmap
Metric thay đổi theo category nào?	Bar plot

Ví dụ

```
sns.scatterplot(data=df, x=..., y=..., hue="species")
```

Nên làm

- ▶ Mỗi chart trả lời một câu hỏi.
- ▶ Có title, axis label, unit, legend.
- ▶ Chọn chart theo kiểu dữ liệu.
- ▶ Annotate insight quan trọng.

Tránh

- ▶ Quá nhiều màu không có ý nghĩa.
- ▶ 3D chart không cần thiết.
- ▶ Axis bị cắt gây hiểu sai.
- ▶ Trang trí lẫn át thông tin.

Trong notebook, sinh viên sẽ

- ▶ Vẽ phân phối body mass và các chỉ số mô.
- ▶ So sánh body mass theo species và sex.
- ▶ Vẽ biểu đồ phân tán giữa bill length và bill depth.
- ▶ Vẽ heatmap tương quan cho các cột numeric.
- ▶ Viết một kết luận ngắn từ mỗi biểu đồ.

Notebook đi kèm sẽ dùng chính các chart này để luyện cách đọc insight từ dữ liệu.

Từ dữ liệu thô đến model

Những thói quen cần giữ sau buổi học

NumPy

Array, shape, dtype, vectorization, broadcasting.

Pandas

DataFrame, index, missing values, groupby, merge.

Visualization

Plot để hiểu dữ liệu, không chỉ để làm đẹp báo cáo.

Scientific Python stack là cây cầu giữa dữ liệu thô và model học máy.

- ▶ Dữ liệu đã được inspect: shape, dtypes, missing values.
- ▶ Các feature numeric và categorical đã được hiểu rõ.
- ▶ Missing values được xử lý có lý do.
- ▶ GroupBy/visualization đã kiểm tra pattern chính.
- ▶ Không leak thông tin từ test set vào preprocessing.
- ▶ Dữ liệu cuối cùng đã sẵn sàng ở dạng matrix/tensor phù hợp.

Data leakage: ví dụ tính mean/std trên cả train và test. Đúng ra chỉ fit trên train, rồi dùng cùng tham số để transform test.

Sau buổi này

- ▶ Hoàn thành notebook NumPy/Pandas/Viz.
- ▶ Lặp lại EDA trên một dataset mới.
- ▶ Tự viết checklist data cleaning.

Học tiếp

- ▶ Scikit-learn cho ML truyền thống.
- ▶ PyTorch/TensorFlow cho deep learning.
- ▶ Polars/Spark khi dữ liệu lớn.

Thank You

Acknowledgements

Buổi học kết hợp slide khái niệm với notebook thực hành để đi từ dữ liệu thô đến phân tích và trực quan hóa.